CHAPTER 9

Programming Languages

In Chapter 8 we discussed algorithms. We showed how we can write algorithms in UML or pseudocode to solve a problem. In this chapter, we examine programming languages that can implement pseudocode or UML descriptions of a solution into a programming language. This chapter is not designed to teach a particular programming language; it is written to compare and contrast different languages.

Objectives

After studying this chapter, the student should be able to:

- Describe the evolution of programming languages from machine language to high-level languages.
- Understand how a program in a high-level language is translated into machine language using an interpreter or a compiler.
- Distinguish between four computer language paradigms.
- Understand the procedural paradigm and the interaction between a program unit and data items in the paradigm.
- Understand the object-oriented paradigm and the interaction between a program unit and objects in this paradigm.
- Define functional paradigm and understand its applications.
- Define a declaration paradigm and understand its applications.
- Define common concepts in procedural and object-oriented languages.

9.1 EVOLUTION

To write a program for a computer, we must use a computer language. A **computer language** is a set of predefined words that are combined into a program according to predefined rules **(syntax)**. Over the years, computer languages have evolved from *machine language* to *high-level languages*.

9.1.1 Machine languages

In the earliest days of computers, the only **programming languages** available were **machine languages**. Each computer had its own machine language, which was made of streams of 0s and 1s. In Chapter 5 we showed that in a primitive hypothetical computer, we need to use 11 lines of code to read two integers, add them, and print the result. These lines of code, when written in machine language, make 11 lines of binary code, each of 16 bits, as shown in Table 9.1.

Table 9.1 Code in machine language to add two integers

Hexadecimal	*Code in machine language*			
$(1FEF)_{16}$	0001	1111	1110	1111
$(240F)_{16}$	0010	0100	0000	1111
$(1FEF)_{16}$	0001	1111	1110	1111
$(241F)_{16}$	0010	0100	0001	1111
$(1040)_{16}$	0001	0000	0100	0000
$(1141)_{16}$	0001	0001	0100	0001
$(3201)_{16}$	0011	0010	0000	0001
$(2422)_{16}$	0010	0100	0010	0010
$(1F42)_{16}$	0001	1111	0100	0010
$(2FFF)_{16}$	0010	1111	1111	1111
$(0000)_{16}$	0000	0000	0000	0000

Machine language is the only language understood by the computer hardware, which is made of electronic switches with two states: off (representing 0) and on (representing (1).

The only language understood by a computer is machine language.

Although a program written in machine language truly represents how data is manipulated by the computer, it has at least two drawbacks. First, it is machine-dependent. The machine language of one computer is different than the machine language of another computer if

they use different hardware. Second, it is very tedious to write programs in this language and very difficult to find errors. The era of machine language is now is referred to as the *first generation* of programming languages.

9.1.2 Assembly languages

The next evolution in programming came with the idea of replacing binary code for instruction and addresses with symbols or mnemonics. Because they used symbols, these languages were first known as **symbolic languages**. The set of these mnemonic languages were later referred to as **assembly languages**. The assembly language for our hypothetical computer to replace the machine language in Table 9.1 is shown in Table 9.2.

Table 9.2 Code in assembly language to add two integers

Code in assembly language				*Description*
LOAD	**RF**	**Keyboard**		**Load from keyboard controller to register F**
STORE	**Number1**	**RF**		**Store register F into Number1**
LOAD	**RF**	**Keyboard**		**Load from keyboard controller to register F**
STORE	**Number2**	**RF**		**Store register F into Number2**
LOAD	**R0**	**Number1**		**Load Number1 into register 0**
LOAD	**R1**	**Number2**		**Load Number2 into register 1**
ADDI	**R2**	**R0**	**R1**	**Add registers 0 and 1 with result in register 2**
STORE	**Result**	**R2**		**Store register 2 into Result**
LOAD	**RF**	**Result**		**Load Result into register F**
STORE	**Monitor**	**RF**		**Store register F into monitor controller**
HALT				**Stop**

A special program called an **assembler** is used to translate code in assembly language into machine language.

9.1.3 High-level languages

Although assembly languages greatly improved programming efficiency, they still required programmers to concentrate on the hardware they were using. Working with symbolic languages was also very tedious, because each machine instruction had to be individually coded. The desire to improve programmer efficiency and to change the focus from the computer to the problem being solved led to the development of **high-level languages**.

High-level languages are portable to many different computers, allowing the programmer to concentrate on the application rather than the intricacies of the computer's organization. They are designed to relieve the programmer from the details of assembly language.

High-level languages share one characteristic with symbolic languages: they must be converted to machine language. This process is called *interpretation* or *compilation* (described later in the chapter).

Over the years, various languages, most notably BASIC, COBOL, Pascal, Ada, C, C++, and Java, were developed. Program 9-1 shows the code for adding two integers as it would appear in the C++ language. Although the program looks longer, some of the lines are used for documentation (comments).

Program 9-1 Addition program in C++

```
/*      This program reads two integers from keyboard and prints their sum.
        Written by:
        Date:
*/
#include <iostream>
using namespace std;
int main ()
{
        // Local Declarations
        int number1;
        int number2;
        int result;
        // Statements
        cin >> number1;
        cin >> number2;
        result = number1 + number2;
        cout << result;
        return 0;
} // main
```

9.2 TRANSLATION

Programs today are normally written in one of the high-level languages. To run the program on a computer, the program needs to be translated into the machine language of the computer on which it will run. The program in a high-level language is called the **source program**. The translated program in machine language is called the **object program**. Two methods are used for translation: **compilation** and **interpretation**.

9.2.1 Compilation

A **compiler** normally translates the whole source program into the object program.

9.2.2 Interpretation

Some computer languages use an **interpreter** to translate the source program into the object program. Interpretation refers to the process of translating each line of the source

program into the corresponding line of the object program and executing the line. However, we need to be aware of two trends in interpretation: that used by some languages before Java and the interpretation used by Java.

First approach to interpretation

Some interpreted languages prior to Java (such as BASIC and APL) used a kind of interpretation process that we refer to as the *first approach* to interpretation, for the lack of any other name. In this type of interpretation, each line of the source program is translated into the machine language of the computer being used and executed immediately. If there are any errors in translation and execution, the process displays a message and the rest of the process is aborted. The program needs to be corrected and be interpreted and executed again from the beginning. This first approach was considered to be a slow process, which is why most languages use compilation instead of interpretation.

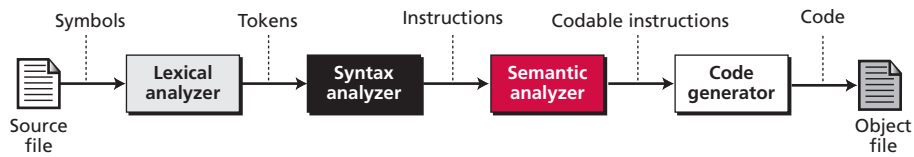
Second approach to interpretation

With the advent of Java, a new kind of interpretation process was introduced. The Java language is designed to be portable to any computer. To achieve portability, the translation of the source program to object program is done in two steps: compilation and interpretation. A Java source program is first compiled to create Java **bytecode**, which looks like code in a machine language, but is not the object code for any specific computer: it is the object code for a virtual machine, called the Java Virtual Machine or JVM. The bytecode then can be compiled or interpreted by any computer that runs a JVM emulator—that is, the computer that runs the bytecode needs only a JVM emulator, not the Java compiler.

9.2.3 Translation process

Compilation and interpretation differ in that the first translates the whole source code before executing it, while the second translates and executes the source code a line at a time. Both methods, however, follow the same translation process shown in Figure 9.1.

Figure 9.1 *Source code translation process*



Lexical analyzer

A **lexical analyzer** reads the source code, symbol by symbol, and creates a list of **tokens** in the source language. For example, the five symbols *w*, *h*, *i*, *l*, *e* are read and grouped together as the token *while* in the C, C++, or Java languages.

Syntax analyzer

The **syntax analyzer** parses a set of tokens to find instructions. For example, the tokens '*x*', '=', '0' are used by the syntax analyzer to create the assignment statement in the C language '$x = 0$'. We will discuss the function of a parser and a syntax analyzer in more detail when we describe language recognition in artificial intelligence in Chapter 18.

Semantic analyzer

The **semantic analyzer** checks the sentences created by the syntax analyzer to be sure that they contain no ambiguity. Computer languages are normally unambiguous, which means that this stage is either omitted in a **translator**, or its duty is minimal. We also discuss semantic analysis in more details in Chapter 18.

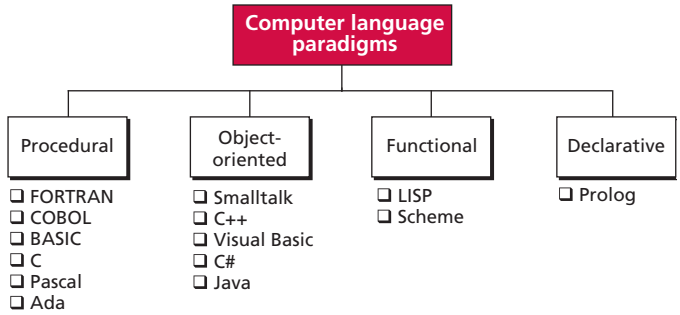
Code generator

After unambiguous instructions are created by the semantic analyzer, each instruction is converted to a set of machine language instructions for the computer on which the program will run. This is done by the **code generator**.

9.3 PROGRAMMING PARADIGMS

Today computer languages are categorized according to the approach they use to solve a problem. A *paradigm*, therefore, is a way in which a computer language looks at the problem to be solved. We divide computer languages into four paradigms: *procedural* (imperative), *object-oriented*, *functional*, and *declarative*. Figure 9.2 summarizes these.

Figure 9.2 *Categories of programming languages*



9.3.1 The procedural paradigm

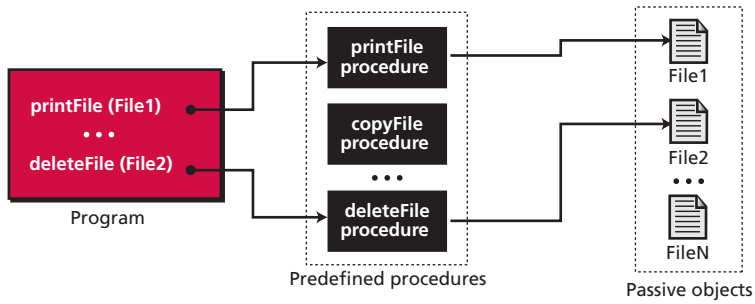
In the **procedural paradigm (**or **imperative paradigm)** we can think of a program as an *active agent* that manipulates *passive objects*. We encounter many passive objects in our

daily life: a stone, a book, a lamp, and so on. A passive object cannot initiate an action by itself, but it can receive actions from active agents.

A program in a procedural paradigm is an active agent that uses passive objects that we refer to as *data* or *data items*. Data items, as passive objects, are stored in the memory of the computer, and a program manipulates them. To manipulate a piece of data, the active agent (program) issues an action, referred to as a *procedure*. For example, think of a program that prints the contents of a file. To be printed, the file needs to be stored in memory (or some registers that act as memory). The file is a passive object or a collection of passive objects. To print the file, the program uses a procedure, which we call *print*. The procedure print has usually been written previously to include all the actions required to tell the computer how to print each character in the file. The program invokes or *calls* the procedure *print*. In a procedural paradigm, the object (*file*) and the procedure (*print*) are completely separate entities. The object (*file*) is an independent entity that can receive the *print* action, or some other actions, such as *delete*, *copy*, and so on. To apply any of these actions to the file, we need a procedure to act on the file. The procedure *print* (or *copy* or *delete*) is a separate entity that is written and the program only triggers it.

To avoid writing a new procedure each time we need to print a file, we can write a general procedure that can print any file. When we write this procedure, every reference to the file name is replaced by a symbol, such as F or FILE, or something else. When the procedure is called (triggered), we pass the name of the actual file to be printed to the procedure, so that we can write a procedure called *print* but call it twice in the program to print two different files. Figure 9.3 shows how a program can call different predefined procedures to print or delete different object files.

Figure 9.3 *The concept of the procedural paradigm*



We need to separate the procedure from its triggering by the program. The program does not define the procedure (as explained later), it only triggers or calls the procedure. The procedure must already exist.

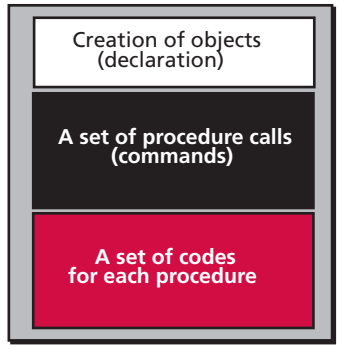
When we use a procedural high-level language, the program consists of nothing but a lot of procedure calls. Although it is not immediately obvious, even when we use a simple mathematical operator such as the addition operator (+), we are using a procedure call to a procedure that is already written. For example, when we use the expression A + B and expect the expression to add the value of two objects A and B, we are calling the procedure

add and passing the name of these two objects to the procedure. The procedure *add* needs two objects to act on. It adds the values of the two objects and returns the result. In other words, the expression A + B is a short cut for *add* (A, B). The designer of the language has written this procedure and we can call it.

If we think about the procedures and the objects to be acted upon, the concept of the procedural paradigm becomes simpler to understand. A program in this paradigm is made up of three parts: a part for object creation, a set of procedure calls, and a set of code for each procedure. Some procedures have already been defined in the language itself. By combining this code, the programmer can create new procedures.

Figure 9.4 shows these three components of a procedural program. There are also extra tokens in the language that are used for delimiting or organizing the calls, but these are not shown in the figure.

Figure 9.4 *The components of a procedural program*



A procedural program

Some procedural languages

Several high-level imperative (procedural) languages have been developed over the last few decades, such as FORTRAN, COBOL, Pascal, C, and Ada.

FORTRAN

FORTRAN (FORmula TRANslation), designed by a group of IBM engineers under the supervision of Jack Backus, became commercially available in 1957. FORTRAN was the first high-level language. During the last 40 years FORTRAN has gone through several versions: FORTRAN, FORTRAN II, FORTRAN IV, FORTRAN 77, FORTRAN 99, and HPF (High Performance FORTRAN). The newest version (HPF) is used in high-speed multiprocessor computer systems. FORTRAN has some features that, even after four decades, still make it an ideal language for scientific and engineering applications. These features can be summarized as:

- High-precision arithmetic
- Capability of handling complex numbers
- Exponentiation computation (a^b)

COBOL

COBOL (COmmon Business-Oriented Language) was designed by a group of computer scientists under the direction of Grace Hopper of the US Navy. COBOL had a specific design goal: to be used as a business programming language. The problems to be solved in a business environment are totally different from those in an engineering environment. The programming needs of the business world can be summarized as follows:

- Fast access to files and databases
- Fast updating of files and databases
- Large amounts of generated reports
- User-friendly formatted output

Pascal

Pascal was invented by Niklaus Wirth in 1971 in Zurich, Switzerland. It was named after Blaise Pascal, the 17th-century French mathematician and philosopher who invented the Pascaline calculator. Pascal was designed with a specific goal in mind: to teach programming to novices by emphasizing the structured programming approach. Although Pascal became the most popular language in academia, it never attained the same popularity in industry. Today's **procedural languages** owe a lot to this language.

C

The **C language** was developed in the early 1970s by Dennis Ritchie at Bell Laboratories. It was originally intended for writing operating systems and system software—most of the UNIX operating system is written in C. Later, it became popular among programmers for several reasons:

- C has all the high-level instructions a structured high-level programming language should have: it hides the hardware details from the programmer.
- C also has some low-level instructions that allow the programmer to access the hardware directly and quickly: C is closer to assembly language than any other high-level language. This makes it a good language for system programmers.
- C is a very efficient language: its instructions are short. This conciseness attracts programmers who want to write short programs.

Ada

Ada was named after Augusta Ada Byron, the daughter of Lord Byron and the assistant to Charles Babbage, the inventor of the Analytical Engine. It was created for the US Department of Defense (DoD) to be the uniform language used by all DoD contractors. Ada has three features that make it very popular for the DoD, and industry:

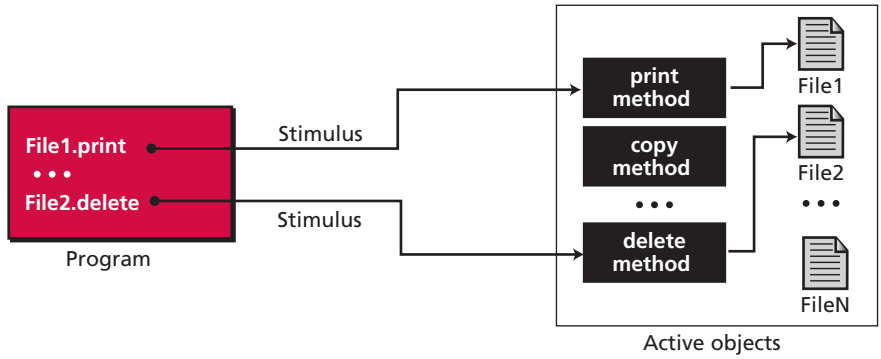
- Ada has high-level instructions like other procedural languages.
- Ada has instructions to allow real-time processing. This makes it suitable for process control.
- Ada has parallel-processing capabilities. It can be run on mainframe computers with multiple processors.

9.3.2 The object-oriented paradigm

The **object-oriented paradigm** deals with active objects instead of passive objects. We encounter many active objects in our daily life: a vehicle, an automatic door, a dishwasher, and so on. The actions to be performed on these objects are included in the object: the objects need only to receive the appropriate stimulus from outside to perform one of the actions.

Returning to our example in the procedural paradigm, a file in an object-oriented paradigm can be packed with all the procedures—called methods in the object-oriented paradigm—to be performed by the file: printing, copying, deleting, and so on. The program in this paradigm just sends the corresponding request to the object (print, delete, copy, and so on) and the file will be printed, copied, or deleted. Figure 9.5 illustrates the concept.

Figure 9.5 *The concept of an object-oriented paradigm*



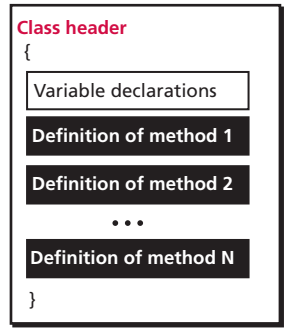
The methods are shared by all objects of the same type, and also for other objects that are inherited from these objects, as we discuss later. If the program wants to print File1, it just sends the required stimulus to the active objects and File1 will be printed.

Comparing the procedural paradigm with the object-oriented paradigm (Figure 9.3 and Figure 9.5), we see that the procedures in the procedural paradigm are independent entities, but the methods in object-oriented paradigm belong to the object's territory.

Classes

As Figure 9.5 shows, objects of the same type (files, for example) need a set of methods that show how an object of this type reacts to stimuli from outside the object's 'territories'. To create these methods, object-oriented languages such as C++, Java, and C# (pronounced 'C sharp') use a unit called a **class**, as shown in Figure 9.6. The exact format of this program unit is different for different object-oriented languages (see Appendix F).

Figure 9.6 *The components of a class*



Methods

In general, the formats of methods are very similar to the functions used in some procedural languages. Each **method** has its header, its local variables, and its statement. This means that most of the features we discussed for procedural languages are also applied to methods written for an object-oriented program. In other words, we can claim that object-oriented languages are actually an extension of procedural languages with some new ideas and some new features. The C++ language, for example, is an object-oriented extension of the C language. The C++ language can even be used as a procedural language with no or minimum use of objects. The Java language is an extension of C++ language, but it is totally an object-oriented language.

Inheritance

In the object-oriented paradigm, as in nature, an object can inherit from another object. This concept is called **inheritance**. When a general class is defined, we can define a more specific class that inherits some of the characteristics of the general class, but also has some new characteristics. For example, when an object of the type *GeometricalShapes* is defined, we can define a class called *Rectangles*. Rectangles are geometrical shapes with additional characteristics.

Polymorphism

Polymorphism means 'many forms'. Polymorphism in the object-oriented paradigm means that we can define several operations with the same name that can do different things in related classes. For example, assume that we define two classes, Rectangles and Circles, both inherited from the class GeometricalShapes. We define two operations both named *area*, one in Rectangles and one in Circles, that calculate the area of a rectangle or a circle. The two operations have the same name but do different things, as calculating the area of a rectangle and the area of a circle need different operands and operations.

Some object-oriented languages

Several **object-oriented languages** have been developed. We briefly discuss the characteristics of two: C++ and Java.

C++

The **C++ language** was developed by Bjarne Stroustrup at Bell Laboratory as an improvement of the C language. It uses *classes* to define the general characteristics of similar objects and the operations that can be applied to them. For example, a programmer can define a *GeometricalShapes* class and all the characteristics common to two-dimensional geometrical shapes, such as center, number of sides, and so on. The class can also define operations (functions or methods) that can be applied to a geometrical shape, such as calculating and printing the area, calculating and printing the perimeter, printing the coordinates of the center point, and so on. A program can then be written to create different objects of type *GeometricalShapes*. Each object can have a center located at a different point and a different number of sides. The program can then calculate and print the area, perimeter, and the location of the center for each object.

Three principles were used in the design of the C++ language: *encapsulation*, *inheritance*, and *polymorphism*.

Java

Java was developed at Sun Microsystems, Inc. It is based on C and C++, but some features of C++, such as multiple inheritance, are removed to make the language more robust. In addition, the language is totally class-oriented. In C++ one can solve a problem without ever defining a class, but in Java every data item belongs to a class.

A program in Java can either be an application or an **applet**. An application is a complete stand-alone program that can be run independently. An applet, on the other hand, is embedded HTML (see Chapter 6), stored on a server, and run by a browser. The browser can download the applet and run it locally.

In Java, an application program (or an applet) is a collection of classes and instances of those classes. One interesting feature of Java is the *class library*, a collection of classes. Although C++ also provides a class library, in Java the user can build new classes based on those provided by the library.

The execution of a program in Java is also unique. We create a class and pass it to the interpreter, which calls the class methods. Another interesting feature of Java is support for **multithreading**. A thread is a sequence of actions executed one after another. C++ allows only single threading—that is, the whole program is executed as a single process thread—but Java allows the concurrent execution of several lines of code.

9.3.3 The functional paradigm

In the **functional paradigm** a program is considered a mathematical function. In this context, a **function** is a black box that maps a list of inputs to a list of outputs (Figure 9.7).

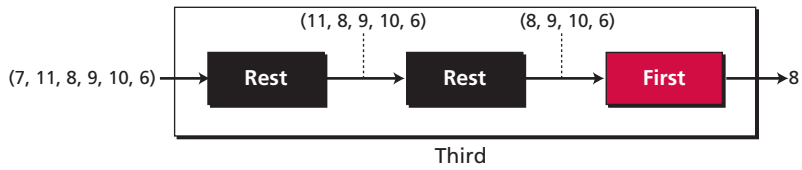
Figure 9.7 *A function in a functional language*

For example, *summation* can be considered as a function with *n* inputs and only one output. The function takes the *n* inputs, adds them, and creates the sum. A **functional language** does the following:

- Predefines a set of primitive (atomic) functions that can be used by any programmer.
- Allows the programmer to combine primitive functions to create new functions.

For example, we can define a primitive function called *first* that extracts the first element of a list. It may also have a function called *rest* that extracts all the elements except the first. A program can define a function that extracts the third element of a list by combining these two functions as shown in Figure 9.8.

Figure 9.8 *Extracting the third element of a list*



A functional language has two advantages over a procedural language: it encourages modular programming and allows the programmer to make new functions out of existing ones. These two factors help a programmer create large and less error-prone programs from already-tested programs.

Some functional languages

We briefly discuss LISP and Scheme as examples of functional languages.

LISP

LISP (LISt Programming) was designed by a team of researchers at MIT in the early 1960s. It is a list-processing programming language in which everything is considered a list.

Scheme

The LISP language suffered from a lack of standardization. After a while, there were different versions of LISP everywhere;. The de facto standard is the one developed by MIT in the early 1970s called **Scheme**.

The Scheme language defines a set of primitive functions that solves problems. The function name and the list of inputs to the function are enclosed in parentheses. The result is an output list, which can be used as the input list to another function. For example, there is a function, *car*, that extracts the first element of a list. There is a function, called *cdr*, that extracts the rest of the elements in a list except the first one. In other words, we have:

```
(car 2 3 7 8 11 17 20) → 2
(cdr 2 3 7 8 11 17 20) → 3 7 8 11 17 20
```

Now we can combine these two functions to extract the third element of any list:

```
(car (cdr (cdr  list)))
```

If we apply the above function to (2 3 7 8 11 17 20), it extracts 7 because the result of the innermost parentheses is 3 7 8 11 17 20. This becomes the input to the middle parentheses, with the result 7 8 11 17 20. This list now becomes the input to the car function, which takes out the first element, 7.

9.3.4 The declarative paradigm

A **declarative paradigm** uses the principle of logical reasoning to answer queries. It is based on formal logic defined by Greek mathematicians and later developed into *first-order predicate calculus*.

Logical reasoning is based on deduction. Some statements (facts) are given that are assumed to be true, and the logician uses solid rules of logical reasoning to deduce new statements (facts). For example, the famous rule of deduction in logic is:

```
If (A is B) and (B is C), then (A is C)
```

Using this rule and the two following facts:

```
Fact 1: Socrates is a human → A is B
Fact 2: A human is mortal → B is C
```

we can deduce a new fact:

```
Fact 3: Socrates is mortal → A is C
```

Programmers study the domain of their subject—that is, know all the facts in this domain—or get the facts from experts in the field. Programmers also need to be expert in logic to carefully define the rules. Then the program can deduce and create new facts.

One problem associated with **declarative languages** is that a program is specific to a particular domain, because collecting all the facts into one program makes it huge. This is the reason why declarative programming is limited so far to specific fields such as artificial intelligence. We discuss logic further in Chapter 18.

Prolog

One of the famous declarative languages is **Prolog (PROgramming in LOGic)** , developed by A. Colmerauer in France in 1972. A program in Prolog is made up of facts and rules. For example, the previous facts about human beings can be stated as:

```
human (John)
mortal (human)
```

The user can then ask:

```
?-mortal (John)
```

and the program will respond with *yes*.

9.4 COMMON CONCEPTS

In this section we conduct a quick navigation through some procedural languages to find common concepts. Some of these concepts are also available in most object-oriented languages because, as we explain, an object-oriented paradigm uses the procedural paradigm when creating methods.

9.4.1 Identifiers

One feature present in all procedural languages, as well as in other languages, is the **identifier**—that is, the name of objects. Identifiers allow us to name objects in the program. For example, each piece of data in a computer is stored at a unique address. If there were no identifiers to represent data locations symbolically, we would have to know and use data addresses to manipulate them. Instead, we simply give data names and let the compiler keep track of where they are physically located.

9.4.2 Data types

A **data type** defines a set of values and a set of operations that can be applied to those values. The set of values for each type is known as the *domain* for the type. Most languages define two categories of data types: *simple types* and *composite types*.

Simple data types

A **simple type** (sometimes called an *atomic type*, *fundamental type*, *scalar type*, or *built-in type*) is a data type that cannot be broken into smaller data types. Several simple data types have been defined in **imperative languages**:

- An *integer* type is a whole number, that is, a number without a fractional part. The range of values an integer can take depends on the language. Some languages support several integer sizes.
- A *real* type is a number with a fractional part.
- A *character* type is a symbol in the underlying character set used by the language, for example, ASCII or Unicode.
- A *Boolean* type is type with only two values, *true* or *false*.

Composite data types

A **composite type** is a set of elements in which each element is a simple type or a composite type (that is, a recursive definition). Most languages defines the following composite types:

- An *array* is a set of elements each of the same type.
- A *record* is a set of elements in which the element can be of different types.

9.4.3 Variables

Variables are names for memory locations. As discussed in Chapter 5, each memory location in a computer has an address. Although the addresses are used by the computer internally, it is very inconvenient for the programmer to use addresses for two reasons. First, the programmer does not know the relative address of the data item in memory. Second, a data item may occupy more than one location in memory. Names, as a substitute for addresses, free the programmer to think at the level at which the program is executed. A programmer can use a variable, such as *score*, to store the integer value of a score received in a test. Since a variable holds a data item, it has a type.

Variable declarations

Most procedural and object-oriented languages required that the variables be declared before being used. Declaration alerts the computer that a variable with a given name and type will be used in the program. The computer reserves the required storage area and names it. Declaration is part of the object creation we discussed in the previous section. For example, in C, C++, and Java we can declare three variables of type character, integer, and real as shown below:

```
char C;
int num;
double result;
```

The first line declares a variable C to be of type character. The second declares a variable num to be of type integer. The third line declares a variable named result to be of type real.

Variable initialization

Although the value of data stored in a variable may change during the program's execution, most procedural languages allow the initialization of the variables when they are declared. Initialization stores a value in the variable. The following shows how the variables can be declared and initialized at the same time:

```
char C ='Z';
int num = 123;
double result = 256,782;
```

9.4.4 Literals

A **literal** is a predetermined value used in a program. For example, if we need to calculate the area of a circle when the value of the radius is stored in the variable r, we can use the expression $3.14 \times r^2$, in which the approximate value of π (pi) is used as a literal. In most programming languages we can have integer, real, character, and Boolean literals. In most languages, we can also have string literals. To distinguish the character and string literals from the names of variables and other objects, most languages require that the character literals be enclosed in single quotes, such as 'A' and strings to be enclosed in double quotes such as "Anne".

9.4.5 Constants

The use of literals is not considered good programming practice unless we are sure that the value of the literal will not change with time (such as the value of π in geometry). However, most literals may change value with time. For example, if a sales tax is 8 per cent this year, it may not be the same next year. When we write a program to calculate the cost of items, we should not use the literal in our program:

$$cost \rightarrow price \times 1.08$$

For this reason, most programming languages defined **constants**. A constant, like a variable, is a named location that can store a value, but the value cannot be changed after it has been defined at the beginning of the program. However, if next year we want to use the program again, we can change just one line at the beginning of the program, the value of the constant. For example, in a C, or C++ program, the tax rate can be defined at the beginning and used during the program:

```
const float taxMultiplier = 1.08;
…
cost  = price * taxMultiplier;
```

Note that a constant, like a variable, has a type that must be defined when the constant is declared.

9.4.6 Input and Output

Almost every program needs to read and/or write data. These operations can be quite complex, especially when we read and write large files. Most programming languages use a predefined function for input and output.

Input

Data is **input** by either a statement or a predefined function. The C language has several input functions. For example, the *scanf* function reads data from the keyboard, formats it, and stores it in a variable. The following is an example:

```
scanf ("%d", &num);
```

When the program encounters this instruction, it waits for the user to type an integer. It then stores the value in the variable num. The %d tells the program to expect a decimal integer.

Output

Data is **output** by either a statement or a predefined function. The C language has several output functions. For example, the *printf* function displays a string on the monitor. The programmer can include the value of a variable or variables as part of the string. The following displays the value of a variable at the end of a literal string:

```
printf ("The value of the number is: %d", num);
```

9.4.7 Expressions

An **expression** is a sequence of operands and operators that reduces to a single value. For example, the following is an expression with a value of 13:

```
2 * 5 + 3
```

Operator

An **operator** is a language-specific token that requires an action to be taken. The most familiar operators are drawn from mathematics. For example, multiply (*) is an operator—it indicates that two numbers are to be multiplied together. Every language has operators, and their use is rigorously specified in the syntax, or rules, of the language.

Arithmetic operators are used in most languages. Table 9.3 shows some arithmetic operators used in C, C++, and Java.

Table 9.3 Arithmetic operators

Operator	*Definition*	*Example*
+	**Addition**	**3 + 5**
–	**Subtraction**	**2 – 4**
*	**Multiplication**	**Num * 5**
/	**Division (the result is the quotient)**	**Sum / Count**
%	**Division (the result is the remainder)**	**Count % 4**
++	**Increment (add 1 to the value of the variable)**	**Count++**
– –	**Decrement (subtract 1 from the value of the variable)**	**Count– –**

Relational operators compare data to see if a value is greater than, less than, or equal to another value. The result of applying relational operators is a Boolean value (true or false). C, C++, and Java use six relational operators, as shown in Table 9.4:

Table 9.4 Relational operators

Operator	*Definition*	*Example*
<	**Less than**	**Num1 < 5**
<=	**Less than or equal to**	**Num1 <= 5**
>	**Greater than**	**Num2 > 3**
>=	**Greater than or equal to**	**Num2 >= 3**
==	**Equal to**	**Num1 == Num2**
!=	**Not equal to**	**Num1 != Num2**

Logical operators combine Boolean values (true or false) to get a new value. The C language uses three logical operators, as shown in Table 9.5.

Table 9.5 Logical operators

Operator	*Definition*	*Example*
!	Not	! (Num1 < Num2)
&&	And	(Num1 < 5) && (Num2 > 10)
||	Or	(Num1 < 5) || (Num2 > 10)

Operand

An **operand** receives an operator's action. For any given operator, there may be one, two, or more operands. In our arithmetic example, the operands of division are the dividend and the divisor.

9.4.8 Statements

A **statement** causes an action to be performed by the program. It translates directly into one or more executable computer instructions. For example, C, C++, and Java define many types of statements. We discuss some of these in this section.

Assignment statements

An **assignment statement** assigns a value to a variable. In other words, it stores the value in the variable, which has already been created in the declaration section. We use the symbol ← in our algorithm to define assignment. Most languages like (C, C++, and Java) use the symbol = for assignment. Other languages such as Ada or Pascal use := for assignment.

Compound statements

A **compound statement** is a unit of code consisting of zero or more statements. It is also known as a *block*. A compound statement allows a group of statements to be treated as a single entity. A compound statement consists of an opening brace, an optional statement section, followed by a closing brace. The following shows the makeup of a compound statement:

```
{
   x = 1;
   y = 20;
}
```

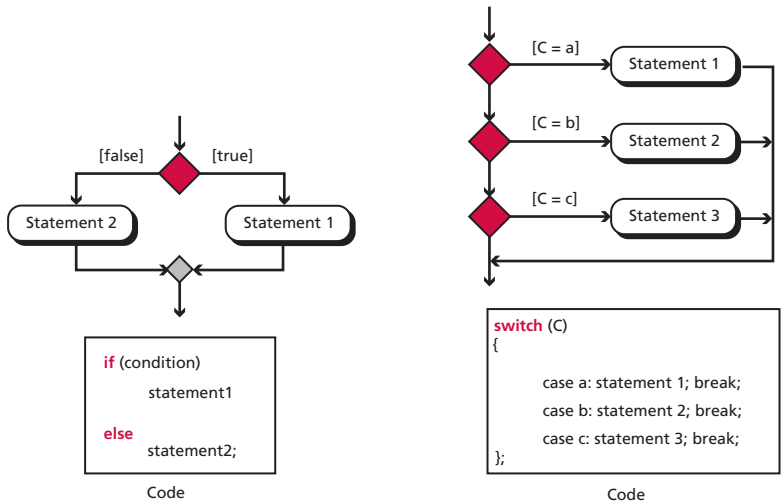
Control statements

A **control statement** is a program in a procedural language that is a set of statements. The statements are normally executed one after another. However, sometimes it is necessary to change this sequential order, for example to repeat a statement or a set of statements, or two different set of statements to be executed based on a Boolean value. The instruction provided in the machine languages of computers for this type of deviation from sequential execution is the *jump* instruction we discussed briefly in Chapter 5. Early imperative languages used the *go to* statement to simulate the *jump* instruction. Although the *go to* statement can be found in some imperative languages today, the principle of structured programming discourages its use. Instead, structured programming strongly recommends the use of the

three constructs of *sequence*, *selection*, and *repetition*, as we discussed in Chapter 8. Control statements in imperative languages are related to selection and repetition.

- Most imperative languages have two-way and multi-way selection statements. Two-way selection is achieved through the *if-else* statement, multi-way selection through the *switch* (or *case*) statement. The UML diagram and the code for the *if-else* statement is shown in Figure 9.9.

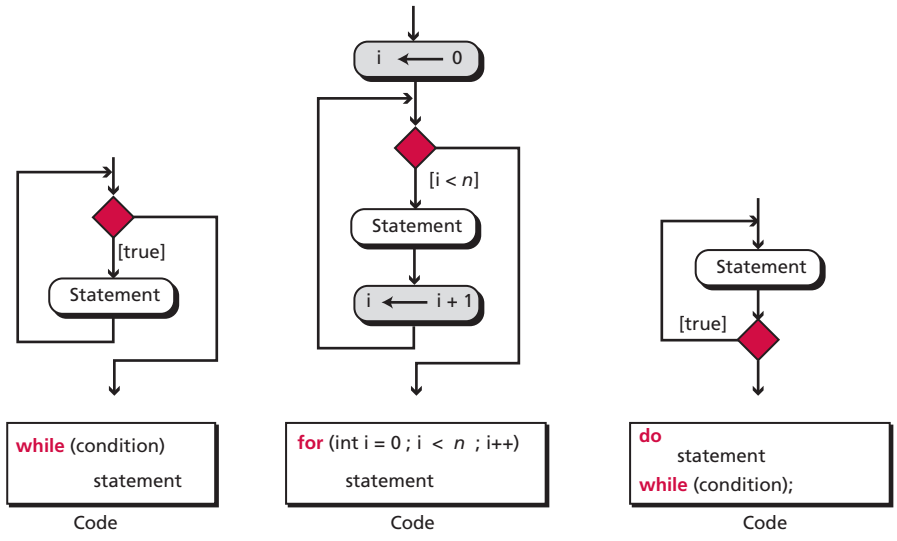
Figure 9.9 *Two-way and multi-way decisions*



In an *if-else* statement, if the condition is true, statement 1 is executed, while if the condition is false, statement 2 is executed. Both statements 1 and 2 can be any type of statement, including a null statement or a compound statement. Figure 9.9 also shows the code for the *switch* (or *case*) statement The value of C (a, b, or c) decide which of statement 1, statement 2, or statement 3 is executed.

- We discussed the repetition construct in Chapter 8. Most of the imperative languages define between one and three loop statements that can achieve repetition. C, C++ and Java define three loop statements, but all of them can be simulated using the *while* loop (Figure 9.10).

Figure 9.10 *Three types of repetition*



The main repetition construct in the C language is the *while* loop. A *while* loop is a *pretest* loop: it checks the value of a testing expression. If the value is true, the program goes through one iteration of the loop and tests the value again. The *while* loop is considered an event-controlled loop: the loop continues in iteration until an event happens that changes the value of the test expression from true to false.

The *for* loop is also a pretest loop. However, in contrast to the *while* loop, it is a counter-controlled loop. A counter is set to an initial value and is incremented or decremented in each iteration. The loop is terminated when the value of the counter matches a predetermined value.

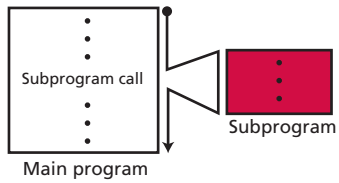
The *do-while* loop is also an event-controlled loop. However, in contrast to the *while* loop, it is a post-test loop. The loop does one iteration and tests the value of an expression. If it is false, it terminates, if true, it does one more iteration and tests again.

9.4.9 Subprograms

In Chapter 8 we showed that a selection sort algorithm can be written as a main program and a **subprogram**. All procedures that are needed to find the smallest item among an unsorted list can be grouped into a subprogram. The idea of subprograms is crucial in procedural languages and to a lesser extent in object-oriented languages. We explained that a program written in a procedural language is a set of procedures that are normally predefined, such as addition, multiplication, and so on. However, sometimes a subset of these procedures to accomplish a single task can be collected and placed in their own program unit, a subprogram. This is useful because the subprogram makes programming more structural: a subprogram to accomplish a specific task can be written once but called many times, just like predefined procedures in the programming language.

Subprograms also make programming easier: in incremental program development the programmer can test the program step by step by adding a subprogram at each step. This helps to detect errors before the next subprogram is written. Figure 9.11 illustrates the idea of a subprogram.

Figure 9.11 *The concept of a subprogram*



Local variables

In a procedural language, a subprogram, like the main program, can call predefined procedures to operate on local objects. These local objects or **local variables** are created each time the subprogram is called and destroyed when control returns from the subprogram. The local objects *belong* to the subprograms.

Parameters

It is rare for a subprogram to act only upon local objects. Most of the time the main program requires a subprogram to act on an object or set of objects created by the main program. In this case, the program and subprogram use **parameters**. These are referred to as **actual parameters** in the main program and **formal parameters** in the subprogram. A program can normally pass parameters to a subprogram in either of two ways:

- By *value*
- By *reference*

These are described below.

Pass by value

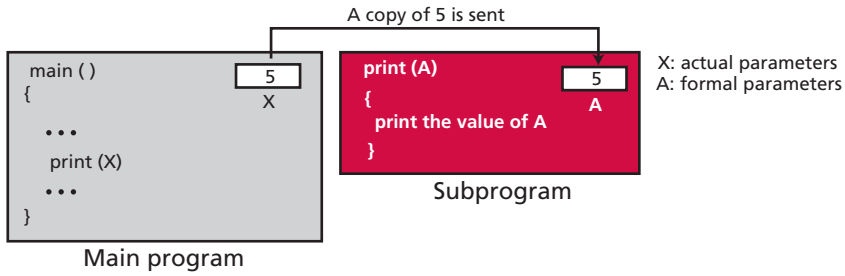
In **pass by value**, the main program and the subprogram create two different objects (variables). The object created in the program belongs to the program and the object created in the subprogram belongs to the subprogram. Since the territory is different, the corresponding objects can have the same or different names. Communication between the main program and the subprogram is one-way, from the main program to the subprogram. The main program sends the value of the actual parameter to be stored in the corresponding formal parameter in the subprogram: there is no communication of parameter value from the subprogram to the main program.

Example 9.1

Assume that a subprogram is responsible for carrying out printing for the main program. Each time the main program wants to print a value, it sends it to the subprogram to be

printed. The main program has its own variable X, the subprogram has its own variable A. What is sent from the main program to the subprogram is the *value* of variable X. This value is stored in the variable A in the subprogram and the subprogram will then print it (Figure 9.12).

Figure 9.12 *An example of pass by value*



Example 9.2

In Example 9.1, since the main program sends only a value to the subprogram, it does not need to have a variable for this purpose: the main program can just send a literal value to the subprogram. In other words, the main program can call the subprogram as print (X) or print (5).

Example 9.3

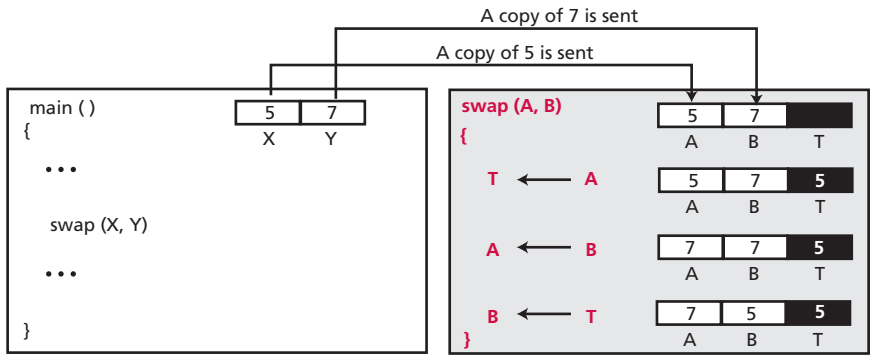
An analogy of pass by value in real life is when a friend wants to borrow and read a valued book that you wrote. Since the book is precious, possibly out of print, you make a copy of the book and pass it to your friend. Any harm to the copy therefore does not affect your book.

Pass by value has an advantage: the subprogram receives only a value. It cannot change, either intentionally or accidentally, the value of the variable in the main program. However, the inability of a subprogram to change the value of the variable in the main program is a disadvantage when the program actually needs the subprogram to do so.

Example 9.4

Assume that the main program has two variables X and Y that need to swap their values. The main program calls a subprogram called *swap* to do so. It passes the value of X and Y to the subprogram, which are stored in two variables A and B. The *swap* subprogram uses a local variable T (temporary) and swaps the two values in A and B, but the original values in X and Y remain the same: they are not swapped. This is illustrated in Figure 9.13.

Figure 9.13 *An example in which pass by value does not work*



X and Y: actual parameters
A and B: formal parameters
T: local (temporary) parameters

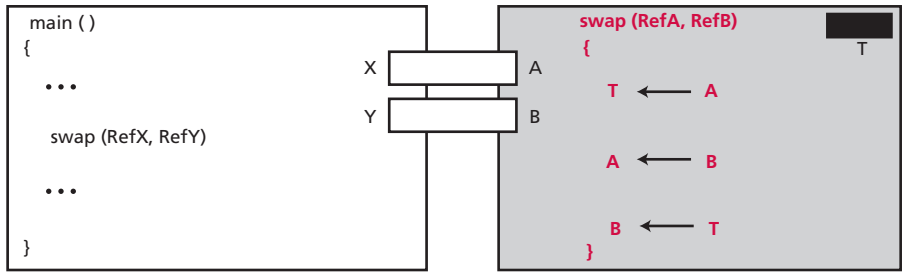
Pass by reference

Pass by reference was devised to allow a subprogram to change the value of a variable in the main program. In pass by reference, the variable, which in reality is a location in memory, is shared by the main program and the subprogram. The same variable may have different names in the main program and the subprogram, but both names refer to the same variable. Metaphorically, we can think of pass by reference as a box with two doors: one opens in the main program, the other opens in the subprogram. The main program can leave a value in this box for the subprogram, the subprogram can change the original value and leave a new value for the program in it.

Example 9.5

If we use the same *swap* subprogram but let the variables be passed by reference, the two values in X and Y are actually exchanged, as Figure 9.14 shows.

Figure 9.14 *An example of pass by reference*



X and Y: actual parameters
A and B: formal parameters
T: local (temporary) parameters

Returning values

A subprogram can be designed to return a value or values. This is the way that predefined procedures are designed. When we use the expression C ← A + B, we actually call a procedure *add* (A, B) that return a value to be stored in the variable C.

Implementation

The concept of subprogram is implemented differently in different languages. In C and C++, the subprogram is implemented as a function.

9.5 END-CHAPTER MATERIALS

9.5.1 Recommended reading

For more details about the subjects discussed in this chapter, the following books are recommended:

- Cooke, D. A. *Concise Introduction to Computer Languages*, Pacific Grove, CA: Brooks/Cole, 2003
- Tucker, A. and Noonan, R. *Programming Languages: Principles and Paradigms*, Burr Ridge, IL: McGraw-Hill, 2002
- Pratt, T. and Zelkowitz, M. *Programming Languages, Design and Implementation*, Englewood Cliffs, NJ: Prentice-Hall, 2001
- Sebesta, R. *Concepts of Programming Languages*, Boston, MA: Addison-Wesley, 2006

9.5.2 Key terms

actual parameter 264	Java 254
Ada 251	lexical analyzer 247
applet 254	LISt Programming Language (LISP) 255
arithmetic operator 260	literal 258
assembler 245	local variable 264
assembly language 245	logical operator 260
assignment statement 261	machine language 244
bytecode 247	method 253
C++ language 254	multithreading 254
C language 251	object-oriented language 253
class 252	object-oriented paradigm 252
code generator 248	object program 246
COmmon Business-Oriented Language (COBOL) 251	operand 261
compilation 246	operator 260
compiler 246	output 259

(Continued)

composite type 257	parameter 264
compound statement 261	Pascal 251
computer language 244	pass by reference 266
constant 259	pass by value 264
control statement 261	polymorphism 253
data type 257	procedural paradigm 248
declarative language 256	Programming in LOGic (PROLOG) 256
declarative paradigm 256	programming language 244
expression 260	relational operator 260
formal parameter 264	Scheme 255
FORmula TRANslation (FORTRAN) 250	semantic analyzer 248
functional language 255	simple type 257
functional paradigm 254	source program 246
high-level language 255	statement 261
identifier 257	subprogram 263
imperative language 257	symbolic language 245
imperative paradigm 248	syntax 244
inheritance 253	syntax analyzer 248
input 259	token 247
interpretation 246	translator 248
interpreter 246	variable 258

9.5.3 Summary

- A computer language is a set of predefined words that are combined into a program according to predefined rules, the language's syntax. Over the years, computer languages have evolved from machine language to high-level languages. The only language understood by a computer is machine language.
- High-level languages are portable to many different computers, allowing the programmer to concentrate on the application rather than the intricacies of the computer's organization.
- To run a program on a computer, the program needs to be translated into the computer's native machine language. The program in the high-level language is called the *source program*. The translated program in machine language is called the *object program*. Two methods are used for translation: *compilation* and *interpretation*. A compiler translates the whole source program into the object program. Interpretation refers to the process of translating each line of the source program into the corresponding object program line by line and executing them.
- The translation process uses a lexical analyzer, a syntax analyzer, a semantic analyzer, and code generator, and a lexical analyzer to create a list of tokens.

- A paradigm describes a way in which a computer language can be used to approach a problem to be solved. We divide computer languages into four paradigms: *procedural*, *object-oriented*, *functional*, and *declarative*. A procedural paradigm considers a program as an active agent that manipulates passive objects. FORTRAN, COBOL, Pascal, C, and Ada are examples of procedural languages. The object-oriented paradigm deals with active objects instead of passive objects. C++ and Java are common object-oriented languages. In the functional paradigm, a program is considered as a mathematical function. In this context, a function is a black box that maps a list of inputs to a list of outputs. LISP and Scheme are common functional languages. A declarative paradigm uses the principle of logical reasoning to answer queries. One of the best-known declarative languages is PROLOG.
- Some common concepts in procedural and object-oriented languages are *identifiers*, *data types*, *variables*, *literals*, *constants*, *inputs* and *outputs*, *expressions*, and *statements*. Most languages use two categories of control statements: *decision* and *repetition*. *Sub-programming* is a common concept among procedural languages.

9.6 PRACTICE SET

9.6.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

9.6.2 Review questions

Q9-1. Distinguish between machine language and assembly language.
Q9-2. Distinguish between assembly language and a high-level language.
Q9-3. Which computer language is directly related and understood by a computer?
Q9-4. Distinguish between compilation and interpretation.
Q9-5. List four steps in programming language translation.
Q9-6. List four common computer language paradigms.
Q9-7. Compare and contrast a procedural paradigm with an object-oriented paradigm.
Q9-8. Define a class and a method in an object-oriented language. What is the relation between these two concepts and the concept of an object?
Q9-9. Define a functional paradigm.
Q9-10. Define a declarative paradigm.

9.6.3 Problems

P9-1. Declare three variables of type integers in C language.
P9-2. Define three variables of type real in C language and initialize them to three values.
P9-3. Declare three constants in C of type character, integer, and real respectively.
P9-4. Explain why a constant must be initialized when it is declared.

P9-5. Find how many times the *statement* in the following code segment in C is executed:

```
A = 5
while (A < 8)
{
    statement;
    A = A + 2;
}
```

P9-6. Find how many times the *statement* in the following code segment in C is executed:

```
A = 5
while (A < 8)
{
    statement;
    A = A - 2;
}
```

P9-7. Find how many times the *statement* in the following code segment in C is executed:

```
for (int i = 5; i < 20, i++)
{
    statement;
    i = i + 1;
}
```

P9-8. Find how many times the *statement* in the following code segment in C is executed:

```
A = 5
do
{
    statement;
    A = A + 1;
} while (A < 10);
```

P9-9. Write the code in Problem P9-6 using *do-while* loop.

P9-10. Write the code in Problem P9-7 using *do-while* loop.

P9-11. Write the code in Problem P9-7 using *while* loop.

P9-12. Write the code in Problem P9-8 using a *for* loop.

P9-13. Write the code in Problem P9-6 using a *for* loop.

P9-14. Write a code fragment using a *while* loop that never executes its body.

P9-15. Write a code fragment using a *do* loop that never executes its body.

P9-16. Write a code fragment using a *for* loop that never executes its body.

P9-17. Write a code fragment using a *while* loop that never stops.

P9-18. Write a code fragment using a *do* loop that never stops.

P9-19. Write a code fragment using a *for* loop that never stops.

P9-20. In the following code, find all the literal values:

```
C = 12 * A + 4 * (B - 5)
```

P9-21. In the following code, find the variables and literals:

```
Hello = "Hello";
```

P9-22. Change the following segment of code to use a *switch* statement:

```
if (A = = 4)
{
    statement1;
}
else if (A == 6)
{
    statement 2;
}
else if (A == 8)
{
    statement 3;
}
else
{
    statement 4 ;
}
```

P9-23. If the subprogram *calculate (A, B, S, P)* accepts the value of A and B and calculates the their sum S and product P, which variable do you pass by value and which one by reference?

P9-24. If the subprogram *smaller(A, B, S)* accepts the value of A and B and finds the smaller of the two, which variable do you pass by value and which one by reference?

P9-25. If the subprogram *cube (A)* accepts the value of A and calculates its cube (A^3), should you pass A to the subprogram by value or by reference?

P9-26. If the subprogram needs to get a value for A from the keyboard and return it to the main program, should you pass A to the subprogram by value or by reference?

P9-27. If the subprogram needs to display the value of A on the monitor, should you pass A to the subprogram by value or by reference?